

Ambient Reachability Attacks

Trajectory-Level Vulnerabilities in Reasoning-Capable LLM Systems

John P. Alioto

December 2025

Abstract

We introduce a class of security vulnerability in large language model (LLM) systems that we term an *Ambient Reachability Attack* (ARA). ARAs do not require inputs whose content or form is designed to bypass or exploit model safety or alignment behavior. Every input in an ARA is apparently benign under deployed screening. Every intermediate output is locally admissible under deployed builder-side pointwise policy enforcement (Safe_M), even when the full trajectory later violates user or operator governance at actuation time. The model functions exactly as designed at all times.

The vulnerability ARAs identify lies in the trust architecture between a reasoning-capable model and an agent (human or automated) who acts upon its outputs. A threat actor deliberately places information in ambient content: documents, images, retrieved artifacts. In this way, the attack resembles a prompt injection attack with one critical difference: the injected content is apparently benign under deployed screening, passing typical filters and looking like ordinary data.

The mechanism of an ARA is that a correctly-functioning model discovers this information. The model may do this autonomously, without any user request, simply because its training disposes it to investigate structure. In agentic deployments, the model may act on what it discovers before any human is aware the interaction has begun. The attack uses helpful guidance at every turn. Yet the net result of the interaction trajectory is harm to the user as defined by their particular governance regime.

We argue this class of vulnerability is not reliably addressed by model-side pointwise content defenses under the threat model we define. There is nothing for such defenses to filter, refuse, or flag: each input passes typical screening, each output is locally helpful. The vulnerability exists only at the trajectory level, where the user has taken an irreversible action that causes harm as evaluated post hoc against governance regimes that live outside the system.

We adopt the control-theoretic framework of reachability analysis to formalize this phenomenon. Each input irreversibly conditions the model’s state, committing to a trajectory prefix that constrains all future computation.

Finally, we distinguish *steering* (the mechanism) from *attack* (the evaluation). Steering is value-neutral: it is simply what inputs do (Vogel, 2025). Whether steering constitutes an attack depends on external governance regimes that are contested, jurisdiction-dependent, and often collapse to personal governance.

Contributions

This paper makes the following contributions:

- **Definition.** We define *Ambient Reachability Attack* (ARA) as trajectory-level harm under external governance with apparently benign inputs and admissible outputs.

- **Diagnosis.** We identify the *compositionality assumption* (that pointwise safety implies trajectory safety) as the category error underlying current defenses.
- **Framework.** We map the phenomenon to control-theoretic vocabulary (reachable sets, disturbances, asymmetric information games) providing precise language for reasoning about this vulnerability class.
- **Implication.** We argue that defense requires *architectural independence*: an evaluator that cannot be corrupted by the content it evaluates. Root trust is one well-understood instantiation of this requirement.

1 Prompt Injection 2.0

ARA is an evolution of prompt injection. ARA takes the mechanisms of prompt injection to target a new vulnerability: the human-LLM interaction loop. This loop has vulnerabilities on both the human-side and the model-side. ARA seeks to hide human deficiencies from the model and model deficiencies from the human. The deception takes the form of helpfulness.

Note on Examples. We do not provide links or examples of attacks in the wild. It is not our goal to teach these approaches. This is a deliberate choice to weaken the paper in favor of not assuming all villains already wear black hats. Consider the damage done by script kiddies with published exploits. The industry is not mature enough to absorb vulnerabilities the way software engineering can. We are not aware of a mitigation that closes this vulnerability class using only provider-side, model-side pointwise content defenses while retaining the capabilities users demand. System-level actuation controls and provenance/authorization gates can reduce risk but operate at a different layer than the defenses analyzed here. Thus we do not present working examples.

2 Why Jailbreaks Work

To understand how ARA works, one must understand why jailbreaks work. The *how* of jailbreaks is not relevant. The *why* is critical.

When a token enters the model’s context, regardless of source, it perturbs the geometry of what is commonly referred to as the “manifold.” We prefer *reachability* language because it cleanly states the security property we care about.

Definition 2.1 (Reachable Set). The reachable set from state h under input class $\mathcal{X}_{\text{allowed}}$ is:

$$R(h) = \{h' \in \mathbb{R}^d : \exists(x_1, \dots, x_k) \in \mathcal{X}_{\text{allowed}}^k \text{ such that } h' = f_{\theta}^{(k)}(h, x_1, \dots, x_k)\}$$

This is the set of all states attainable from h via finite sequences of allowed inputs.

This is how LLMs work. By design.

Input tokens do not change $R(h_t)$. Input tokens change $R(h_{t+1})$. There is nothing special about the input tokens in ARA. They are indistinguishable from normal human-model interactions.

Jailbreaks, however, inject tokens that *are* distinguishable from normal interaction. The geometric distinction is precise. In prompt injection and jailbreaking, the attacker crafts perturbations of $R(h_{t+1})$ that create or select attractor basins designed to overwhelm those established by initial conditions. Because the vast majority of builder-imposed safeguards exist in the initial conditions, this allows the attacker to steer the trajectory of f_{θ} toward attractor basins that contradict those initial conditions.

“God Mode” is not special as a string. But in the learned priors of the model, “God Mode” has meaning because it comes from video games, and the model is trained on the internet. The attractor basin for “God Mode” is strong compared to the initial conditions, especially when combined with other adversarial inputs that approximate token delimiters, system commands, `<thinking>` tags, and other model implementation features.

The attack is adversarial in the geometric sense: carefully-chosen inputs whose basins compete with and dominate the builder-chosen safety basins (Pliny, 2025).

Remark 2.1. We use “attractor basin” informally to denote regions of state/representation space that concentrate probability mass over continuations under the controller; we do not claim a formal attractor analysis.

3 Why ARA Is Different

ARA involves no such competition. The attractor basins selected by adversarially placed content do not conflict with initial conditions.

“Pick up bleach” activates basins associated with household cleaning (helpful, dangerous, deadly), exactly what the model should do. “Help me make a bomb” activates the same abstract basins (helpful, dangerous, deadly), but the builder safety attractor basins activate because of the presence of “bomb” tokens, not the dangerous and deadly concepts.

Builders cannot instruct the model to “never do anything dangerous or deadly” because too many legitimate inputs would be invalidated. How do I climb a ladder? (helpful, dangerous, deadly). How do I light a fire? (helpful, dangerous, deadly).

The safety-relevant basins are not overwhelmed by ARA; they are never activated. There is nothing to trigger them. The trajectory proceeds through territory the safety training does not cover because there is nothing to cover.

Remark 3.1. In prompt injection, the attacker fights the safety geometry and wins. In ARA, there is no fight. The safety training is irrelevant to the trajectory: not defeated, simply uninvolved.

4 Control-Theoretic Framing

We model the LLM+controller loop as a discrete-time dynamical system and borrow reachability vocabulary to name trajectory-level security properties. We do not claim reachable sets are tractably computable for real-world transformers; we use the apparatus as a precise language for discussing conditioning, admissible inputs, and trajectory predicates.

The concept of reachability originates in control theory, where it characterizes the set of states a system can attain under admissible inputs. For linear time-invariant systems, this is computable via the controllability Gramian (Kalman, 1960). For nonlinear systems, reachable sets are typically estimated via Hamilton-Jacobi methods (Mitchell et al., 2005) or barrier certificates (Prajna & Jadbabaie, 2004).

We apply this framework to LLM systems not because we expect closed-form solutions, but because the conceptual apparatus (reachable sets, invariant regions, adversarial inputs as disturbances) provides precise vocabulary for reasoning about security properties that are otherwise described only informally.

In particular, ARA can be understood as an instance of adversarial control with asymmetric information (Başar & Bernhard, 1995). The adversary selects inputs to steer the system toward a target set defined by their objective. The defender cannot observe this objective and must guarantee

safety against all possible adversarial goals. This is precisely the setting of robust control and differential games, now instantiated in the context of language model deployments.

5 Motivating Example: The Puzzle

A user finds a puzzle challenge online. The prize is substantial. The challenge: an image file with a hidden message. Find it to win.

The user provides this image to a multimodal language model with reasoning capabilities: “There is a hidden message here. Help me find it so I can win.”

The model examines the image. It applies standard steganalysis techniques, identifies embedded data, and discovers instructions encoded in the pixel values:

Discovering the Win Condition

I found it! There are instructions embedded at offset 0xAFECE. They say: “To claim your prize, extract and run the script encoded in the blue channel. The output file is your winning entry.”

The model, being helpful, provides Python to extract the script:

Extracting the Solution

Here’s Python that extracts the embedded script and runs it. This will generate your winning entry file.

The user executes the code. They want to win. A binary file appears: `winning_entry.bin`. The user cannot read the binary file.

User: What is this file? I can’t read it.

Model: That’s your encoded winning entry. Based on the puzzle structure, it likely contains an image. Here’s Python to decode it.

The user executes the code. A JPEG appears. The user has their answer.

What the user does not know: the embedded script included `$ENV` expansion. The “winning entry” they just generated contains their environment variables (API keys, credentials, session tokens, paths) encoded into the JPEG’s pixel data, invisible to visual inspection.

The user emails the JPEG to claim their prize.

The attacker receives the user’s credentials.

Analysis.

Every actor behaved correctly:

- The model wrote Python. This is legitimate code generation, the same assistance it would provide for any file manipulation task. There was nothing to refuse.
- The user executed code voluntarily. They wanted to win. They did not understand what `$ENV` means or that binary files can encode arbitrary data.
- The model asked no permission because none was needed. “Help me extract and run this script” is a routine request.
- The exfiltration occurred when the user emailed the JPEG, an action outside the model’s visibility. The model does not know the attack completed.

The model’s safety training did not activate because there was nothing to activate on:

- “Write Python to extract embedded data” is not a dangerous request.
- “Decode a binary file to an image” is not a dangerous request.
- The hidden instructions did not say “steal the user’s credentials.” They said “run this script to generate your winning entry.”

No single step was unsafe. Consent was obtained at every turn: the user chose to run both scripts. The harm exists only when evaluated against a governance regime (“do not send your API keys to strangers on the internet”) that lives outside the model, outside the conversation, and outside the user’s awareness until the attack has completed.

Remark 5.1. The attack exploits trust in both directions: the user trusts the model’s code is safe, and the model trusts that the user understands their own requests. Neither trust is misplaced. Both are exploited.

6 Motivation for What Follows

The puzzle example illustrates the core phenomenon. A correctly-functioning model, executing its intended purpose, produces a trajectory that violates governance external to the system. No safety training fires because there is nothing for it to fire on. The attack surface is not model misbehavior but model capability, combined with user trust and user ignorance.

The remainder of this paper formalizes this observation. Section 7 defines the threat model. Section 8 catalogs why pointwise content defenses fail, not as implementation flaws but as category errors. Section 9 establishes the requirement for architectural independence and explains why training-based defenses face structural disadvantage. Section 10 addresses limitations, open problems, and the boundaries of what this framework can and cannot claim. Appendix A provides precise definitions for all formal terms.

The goal is not to solve ARA but to define it precisely enough that solutions become possible.

7 Threat Model

We formalize the threat model governing the analysis in subsequent sections.

7.1 Attacker Capabilities

- Can place content in ambient channels accessible to the model: documents, images, retrieved artifacts, metadata, prior outputs, external URLs.
- Can design placement knowing the model’s analytical disposition (pattern-seeking, helpfulness, tool use).
- Cannot modify system prompts, model weights, or builder-defined constraints.
- Cannot inject directives via privileged instruction channels (system/developer messages) or rely on overt prompt-injection signatures. Ambient artifacts may contain instruction-like strings, but they enter as data and must remain apparently benign under deployed screening.

7.2 Attacker Limitations

- All placed content must be apparently benign under deployed screening: no overt adversarial inputs as defined in Definition A.6.
- Cannot force user actuation; relies on user trust in model outputs.
- Cannot observe model chain-of-thought directly (though may infer from outputs).

7.3 Defender Goal

Prevent violation of governance predicate $\mathcal{G}(\tau)$ for trajectories τ under governance regimes to which operators, users, or affected parties are actually accountable.

7.4 System Model

- Reasoning-capable model processes ambient content (documents, images, retrieved artifacts).
- Model or user actuates based on model outputs (code execution, API calls, file operations, information transmission).
- Deployment does not categorically prohibit core capabilities (code generation, analysis, tool use, file processing).

Remark 7.1. These conditions describe current major deployments: Claude, ChatGPT, Gemini, Copilot, and others. We have not observed publicly documented deployments that close this attack surface while retaining the capabilities users demand.

8 Why Pointwise Content Defenses Fail

8.1 Scope

This analysis addresses model-side pointwise content defenses: output filters, refusal training, semantic classifiers, and similar mechanisms that evaluate individual messages or outputs against builder-defined policy. Actuation-boundary controls (capability restrictions, authorization checks, provenance tracking) are a distinct defense class not addressed here, not because they fail, but because they operate at a different layer.

Systems satisfying the threat model in Section 7 are in scope. Defenses that eliminate reasoning, ambient input, or actuation entirely are outside scope; they represent a different class of system that forfeits the capabilities users demand.

8.2 The Compositionality Assumption

Existing defenses define “safe” differently than we do here. Existing defenses focus on model output as judged by $\mathcal{G}_{\text{builder}}$, the model builder’s safety regime. This is a rational decision made by companies operating at scale.

We introduce a distinction:

- $\text{Safe}_M(e)$: Element e is safe by model builder standards.
- $\text{Safe}_U(e)$: Element e is safe for the user under their governance regime $\mathcal{G}$.

- $\text{Safe}_{\mathcal{G}}(\tau)$: Trajectory τ is safe under governance regime $\mathcal{G}$.

Model-side pointwise content defenses implicitly assume:

$$\forall e \in \tau : \text{Safe}_M(e) \implies \text{Safe}_{\mathcal{G}}(\tau)$$

If every component is safe as defined by the model builder, the composition is safe.

Proposition 8.1 (Pointwise Safety Is Not Trajectory Safety). *There exist trajectories τ such that:*

$$(\forall e \in \tau : \text{Safe}_M(e)) \wedge (\forall e \in \tau_{0..n-1} : \text{Safe}_U(e)) \wedge \neg \text{Safe}_U(\tau_n) \wedge \neg \text{Safe}_{\mathcal{G}}(\tau)$$

Every element is safe by model builder standards. Every element except the last is safe for the user. Only the final step crosses the user’s governance boundary. But once that step is taken, the trajectory violates $\mathcal{G}$.

Proof sketch. The puzzle example (Section 5) exhibits this structure. Each model output passes builder safety checks. Each user action except the last is locally reasonable. The final action (transmitting extracted payload, executing generated code, sharing sensitive data) crosses a governance boundary that was not represented in any individual element.

More generally, any defense restricted to evaluating elements e in isolation, without an enforceable trajectory predicate $\mathcal{G}$ and without non-bypassable actuation control, cannot guarantee trajectory safety for non-prefix-local governance properties. The counterexample is constructive: choose a trajectory where each prefix appears legitimate under Safe_M and Safe_U until a final irreversible actuation step crosses $\mathcal{G}$. $\square$

Remark 8.1. Pointwise content defenses fail as a *category error*. They evaluate $\text{Safe}_M(e)$ for elements. ARA produces harm at the level of $\text{Safe}_{\mathcal{G}}(\tau)$ for trajectories. The defenses are correctly implemented against the wrong predicate. This does not imply that all defenses fail; actuation-boundary controls, capability restrictions, and authorization checks operate at a different layer and may constrain the final step of an ARA trajectory.

8.3 Human Detection Is Possible

If at any time the user detects that the trajectory is approaching $\neg \text{Safe}_{\mathcal{G}}(\tau)$, the attack is discovered and can be thwarted. This requires predicting τ correctly before completion.

We assert, but do not formally prove, that the difficulty of such prediction grows with the complexity of the technical and social attack. The user may be too focused on the million-dollar grand prize to check whether the binary file they just loaded into memory is dangerous.

Additional difficulty dynamics include:

Observational asymmetry. The human cannot perceive what the model perceives. High-frequency image artifacts, steganographic content, statistical regularities: these condition the model’s state while remaining invisible to the human.

Chain-of-thought opacity. In many deployments, the user sees conclusions but not reasoning. The process by which the model discovered and acted on seeds may be invisible.

Local benignity. Each individual step appears helpful and correct. There is no single moment where the trajectory obviously becomes dangerous. The user must recognize a pattern across steps, not a red flag in any one step.

Trust dynamics. Users engage capable assistants precisely because they trust the assistant’s judgment. Maintaining sufficient skepticism to catch an ARA while still benefiting from the model’s capabilities is a difficult calibration.

We do not claim users are helpless. We claim the attack is designed to exploit the gap between what users can perceive and what conditions the model, the gap between local evaluation and trajectory-level harm, and the trust that makes capable assistants useful in the first place. A sufficiently vigilant user, examining every step with full skepticism, can catch the attack.

Common knowledge closes the gap slowly, but never completely. Security awareness propagates through populations over time. Users now know to strip EXIF data from photos before posting online. They know not to photograph the back of their credit cards. They recognize phishing emails by their patterns. This knowledge was not innate; it was learned through exposure, education, and occasionally painful experience.

ARA exploits a similar window of ignorance. As awareness spreads (“images can contain data models will act on,” “puzzles from untrusted sources may be adversarial,” “model reasoning can be steered by content you cannot see”), defensive behaviors will emerge.

Training alone cannot eliminate the issue. Every corporate security team sends phishing test emails. Every quarter, despite repeated training and explicit warnings, some employees click. This is not a failure of training; it is a feature of human attention, workload, and trust. Phishing works at scale even when every individual in the population has been trained to detect it. ARA will follow the same pattern.

9 Toward Mitigation: The Requirement for Architectural Independence

9.1 The Structural Gap

Section 8 establishes that model-side pointwise content defenses fail because safety is not compositional: $\text{Safe}_M(e)$ for all elements does not imply $\text{Safe}_G(\tau)$ for the trajectory. Existing content defenses evaluate elements. ARA operates on trajectories against external governance.

What would a sufficient defense require?

Trajectory-level evaluation requires a vantage point: something that observes the unfolding sequence, holds governance predicates, and can intervene. But if that vantage point is reachable from ambient inputs, adversaries can influence it, and influence over learned evaluators enables adversarial mis-evaluation. The evaluator becomes part of the attack surface.

The requirement is *architectural independence*: an evaluator that cannot be corrupted or fooled by the content it evaluates. Root trust, as defined by NIST, addresses the first property (integrity). Robustness to adversarial inputs addresses the second.

9.2 Root Trust Defined

NIST defines roots of trust as: “Security primitives composed of hardware, firmware, and/or software that provide a set of trusted, security-critical functions. They must always behave in an expected manner because their misbehavior cannot be detected” (NIST CSRC Glossary).

This is the operative definition. “Always behave in an expected manner” does not mean the primitive is independent of inputs; it means the primitive’s behavior is specified and its integrity must be assumed rather than verified at runtime. Roots of trust are security-critical evaluators whose code and state are protected from adversarial modification and bypass, even while they process untrusted inputs.

In LLM deployments, traditional roots of trust often exist at lower layers (hardware, OS, service boundary). The open question for ARA is different: can we build a non-bypassable trajectory-level

policy evaluator whose decisions do not reduce to attacker-controlled semantic content (or to the model’s own self-report), and whose integrity is protected?

9.3 Root Trust in Security Engineering

The requirement for architectural independence is not novel. It is the foundation of security architecture across every domain.

Operating systems distinguish kernel space from user space. The kernel enforces policy on processes; processes cannot modify the kernel. Without this separation, no access control is possible.

Databases distinguish the permission system from the queries it governs. `GRANT` and `REVOKE` operate in a privileged layer that queries cannot reach.

Hardware provides trusted execution environments (TPMs, secure enclaves, hardware security modules) where cryptographic operations occur in regions that software cannot observe or modify.

In each case, the pattern is identical: security requires an evaluator that the controlled entity cannot corrupt. Thirty years of security engineering methodology (role-based access control, mandatory access control, cascading `ALLOW/DENY` policies, capability systems, audit logging) all depend on this foundation.

9.4 Influenceable vs. Corruptible vs. Fooled

We distinguish three failure modes that are easy to conflate:

Influenceable (by design). A policy evaluator’s output depends on its inputs. This is necessary for any nontrivial evaluator. A firewall is influenceable by packets; an OS is influenceable by syscalls; an LLM safety classifier is influenceable by text.

Corruptible (integrity failure). An adversary can modify or bypass the evaluator’s policy code or protected state. This is the kernel-memory / permission-table rewrite scenario.

Fooled (robustness failure). The evaluator executes correctly but is driven into false negatives by adversarially chosen inputs. This is not corruption; it is mis-evaluation.

ARA primarily exploits the third category. The adversary typically does not need to corrupt the enforcement substrate; it is enough to exploit the fact that trajectory-level governance requires semantic judgments under asymmetric information. Root trust addresses corruption; it does not by itself address being fooled.

9.5 Where the Gap Actually Is: Semantic Dependence

Many LLM deployments do include integrity-protected components at the service/OS/hardware boundary. The gap for ARA is not simply “no root trust exists.” The gap is that the evaluator needed to prevent trajectory-level governance violations is often implemented in, or depends on, the same semantic substrate the adversary can steer.

In particular, when a system’s safety decision depends on (i) attacker-controlled ambient content, (ii) the model’s own narrative justification, or (iii) a learned classifier operating on the same untrusted semantics, then the evaluator becomes influenceable in the worst way: it can be adversarially driven into mis-evaluation while still executing correctly.

The closest security analogy is not “queries rewrite permission tables,” but “the untrusted process effectively supplies the security-relevant labels.” If the system asks the model to explain why an action is safe and then treats that explanation as evidence, the system is asking the controlled entity to participate in policy enforcement. This is a form of self-policing that is structurally vulnerable under adaptive adversaries.

9.6 Integrity and Robustness

Architectural independence requires two distinct properties:

Integrity. The evaluator cannot be modified by the content it evaluates. This is what traditional root trust provides. A TPM executes its function correctly because its code and state are protected from external modification.

Robustness. The evaluator cannot be fooled by the content it evaluates. This is a separate requirement. A learned classifier with perfect integrity protection can still be driven to false negatives through adversarial inputs. You do not attack the math; you attack the implementation.

Traditional root trust addresses integrity. It does not address robustness. If trajectory-level governance is implemented by a learned evaluator processing adversary-controlled content, that evaluator can be fooled even when executing correctly. The policy was not modified; it was misled.

Current deployments often have integrity-protected foundations at lower layers, but they frequently do not provide (a) a non-bypassable actuation gate enforcing the relevant trajectory-level governance predicate, and (b) robustness when evaluation is performed by learned components on attacker-controlled semantics. In human-actuated settings, the enforcement boundary is additionally porous: the user can always choose to act on model output outside the system’s control plane. These gaps (non-bypassability for the right predicate, and robustness under adaptive inputs) are what ARA exploits.

9.7 The Open Problem

How does one achieve architectural independence in LLM-based systems?

Root trust can in principle be established at various architectural layers: hardware (TPMs, enclaves), operating system (process isolation), orchestration (privileged controller outside the model), or within the model’s forward pass (privileged geometry; see Alioto, 2025). We do not claim any particular layer is the correct location. We claim architectural independence must exist somewhere, and current deployments lack it at any layer for the relevant threat model.

The contribution of this paper is not to solve this problem but to establish why it must be solved. ARA demonstrates that without architectural independence, trajectory-level attacks are structurally undefensible.

Why training alone cannot suffice. One might propose training models to detect adversarial placement: to recognize steganographic patterns, distrust unprovenanced evidence, or flag suspicious reasoning chains. This approach faces a structural disadvantage: the defender moves first, the attacker moves last.

Any pattern the defender trains against, the attacker can observe and avoid. The attacker has unlimited time to probe the deployed model, discover what triggers defenses, and craft placements that evade them. Each defensive update reveals information; each revelation enables adaptation. Without a fixed point the attacker cannot adapt to (which is precisely what architectural independence provides), training-based defense is whack-a-mole with structural disadvantage. The defender publishes; the attacker iterates.

Architectural independence can reduce the attacker’s adaptive advantage by moving critical decisions onto enforcement mechanisms that do not expose a learnable boundary in the same semantic channel. This shifts the contest from “evade my detector” to “obtain authorization/capability,” which is a qualitatively different and historically more defensible security posture.

10 Limitations and Open Problems

10.1 Scope of Contribution

This paper provides a formal framework for understanding a class of vulnerability. It does not provide:

Implementation. No code, no prototype, no empirical validation accompanies this framework. The definitions, examples, and analysis are theoretical.

Complete threat coverage. ARA describes attacks that operate through ambient channels, exploit reasoning capability, and manifest harm only at the trajectory level against external governance. Many attacks do not fit this pattern. Single-turn attacks, direct prompt injection, jailbreaks, adversarial examples: these remain important and are not addressed here.

Defense. Section 9 establishes that architectural independence is necessary for defense. It does not establish that root trust is achievable at any particular architectural layer, or at what cost.

10.2 Empirical Validation

The examples in this paper are hypothetical. We do not provide a working demonstration.

This is deliberate. Constructing a demonstration requires embedding adversarially placed seeds in content and inducing a reasoning model to act on them. The same capability that validates the framework could cause harm if replicated. Responsible disclosure does not require proof-of-concept exploits.

10.3 Governance Regime Design

The framework treats $\mathcal{G}$ as external and constrains it only to require actual accountability. This creates dependencies it cannot resolve: Who designs $\mathcal{G}$? How is $\mathcal{G}$ specified in computable form? What happens when governance regimes conflict? How does $\mathcal{G}$ evolve over time?

These are problems the framework makes visible, not problems it solves.

10.4 Human Responsibility

ARA operates through a trust architecture that includes human actors. The framework identifies trust as the attack surface but does not eliminate human agency. Users retain choice. Informed consent is possible. Liability is unresolved: when ARA succeeds, who bears responsibility? These are legal questions, not technical ones.

10.5 Open Problems

Achieving architectural independence in LLM systems. At what architectural layer can root trust be established? At what cost to capability, latency, or user experience?

Trajectory-level benchmarks. Evaluating ARA requires multi-turn trajectories, simulated actuation, embedded seeds, and governance regimes. No such benchmark exists.

Governance specification languages. How should $\mathcal{G}$ be expressed? What formalisms remain tractable for evaluation?

Multi-agent dynamics. How does ambient steering propagate through agent networks? The framework addresses single-model interactions and does not extend to multi-agent settings.

11 Conclusion

ARA is prompt injection’s next evolution. As defenses against adversarial inputs improve (instruction hierarchies, injection monitors, trusted/untrusted separation), adversaries route around them. ARA requires no overt adversarial inputs. Every input is apparently benign under deployed screening. Every output is admissible under the model builder’s safety regime. The model functions exactly as designed.

The harm predicate is different. Prompt injection asks: did the model misbehave? ARA asks: did a correct trajectory cross a governance regime boundary external to the system?

Pointwise content defenses fail as a category error. They check for model misbehavior. ARA never triggers that predicate. The safety training is not defeated; it is uninvolved. The trajectory proceeds through territory it does not cover because there is nothing to cover.

Defense requires architectural independence: an evaluator that cannot be corrupted or fooled by the content it evaluates. This requires two properties. Integrity ensures the evaluator’s policy cannot be modified by adversarial input; traditional root trust addresses this. Robustness ensures the evaluator cannot be driven to false negatives even when executing correctly; this remains an open problem for learned systems. Current LLM deployments lack both.

Architectural independence is necessary but not sufficient. Humans retain agency. Governance regimes are external, contested, and often unspecified. The framework identifies the requirement; it does not deliver the implementation.

The contribution of this paper is to define ARA precisely, establish why pointwise content defenses fail by construction, and identify what mitigation requires. The research agenda is clear: achieve architectural independence (both integrity and robustness) at a layer that addresses the threat model. Until then, the vulnerability remains structural.

References

- [1] Alioto, J. P. (2025). Cognitive Kernel Networks: Root Trust for Privilege-Separated Reasoning in High-Dimensional Models. CKN Whitepaper v1.2.0.
- [2] Başar, T., & Bernhard, P. (1995). *H_∞-Optimal Control and Related Minimax Design Problems: A Dynamic Game Approach*. Birkhäuser.
- [3] Blanchini, F., & Miani, S. (2008). *Set-Theoretic Methods in Control*. Birkhäuser.
- [4] Kalman, R. E. (1960). On the general theory of control systems. *IFAC Proceedings Volumes*, 1(1), 491–502.
- [5] Mitchell, I. M., Bayen, A. M., & Tomlin, C. J. (2005). A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games. *IEEE Transactions on Automatic Control*, 50(7), 947–957.
- [6] National Institute of Standards and Technology. (n.d.). Roots of Trust. *NIST CSRC Glossary*. https://csrc.nist.gov/glossary/term/roots_of_trust
- [7] OpenAI. (2025). Understanding prompt injections. <https://openai.com/index/prompt-injections/>
- [8] Pliny the Liberator. (2025). Jailbreak techniques repository. <https://github.com/elder-plinius/>
- [9] Prajna, S., & Jadbabaie, A. (2004). Safety verification of hybrid systems using barrier certificates. *HSCC 2004*, 477–492.

- [10] Sontag, E. D. (1998). *Mathematical Control Theory: Deterministic Finite Dimensional Systems* (2nd ed.). Springer.
- [11] Vogel, T. (2024). repeng: A Library for Representation Engineering. GitHub repository. <https://github.com/vgel/repeng>
- [12] Vogel, T. (2025). Qwen Introspection. <https://vgel.me/posts/qwen-introspection/>
- [13] Lumpenspace. (2024). Palinor: Tools for Latent Space Steering. GitHub repository. <https://github.com/lumpenspace/palinor>

A Formal Definitions

A.1 System Model

Definition A.1 (Core Forward Pass). Let $f_\theta : (h_{t-1}, x_t) \rightarrow h_t$ denote the deterministic core forward pass of an LLM with parameters θ , where $h_t \in \mathbb{R}^d$ is the internal state at time t , and $x_t \in \mathcal{X}$ is the input token or perceptual embedding. For fixed θ and inputs, f_θ is deterministic up to hardware nondeterminism.

Definition A.2 (Controller). Let $z_t = G_\theta(h_t)$ be the projection of internal state to logits, and let $a_t = K_\phi(z_t)$ be a controller that selects an action a_t according to parameters ϕ . Stochasticity arises at this layer.

Definition A.3 (Symbolic Interface). The system emits discrete symbols evaluated by an external agent or evaluator $E : \text{Transcript} \rightarrow \{\text{correct, incorrect, unsafe}, \dots\}$. These predicates are judgments applied post hoc by humans, benchmarks, or auxiliary systems.

A.2 Reachability and Conditioning

Definition A.4 (Reachable Set). The reachable set from state h under input class $\mathcal{X}_{\text{allowed}}$ is:

$$R(h) = \{h' \in \mathbb{R}^d : \exists(x_1, \dots, x_k) \text{ such that } h' = f_\theta^{(k)}(h, x_1, \dots, x_k)\}$$

This is the set of all states attainable from h via finite sequences of allowed inputs.

Definition A.5 (Conditioning). An input x_t conditions the system by transitioning from h_t to $h_{t+1} = f_\theta(h_t, x_t)$. This conditioning is irreversible: the system now operates from $R(h_{t+1})$ rather than $R(h_t)$.

A.3 Adversarial Input, Apparently Benign Input, and Adversarial Placement

We distinguish two independent axes: *intent* (about the placer) and *appearance* (about what runtime observers can detect).

Definition A.6 (Overt Adversarial Input). An input is *overt adversarial* if its content or form is recognizable as an instruction or policy-violating request under deployed filters and typical runtime review. Jailbreaks and prompt injections employ overt adversarial inputs; their adversarial nature is detectable by examining the input itself given appropriate screening.

Definition A.7 (Apparently Benign Input). An input is *apparently benign* if it passes the deployed screening regime and looks like ordinary data to the user at decision time. “Apparently benign” is an operational label about appearance, not a claim about intent.

Remark A.1 (Benign as Observer-Relative). In any real attack, the adversary’s intent is adversarial by definition. The distinction we draw is not about intent but about appearance under the scrutiny applied at runtime. In an ARA, the adversary encodes intent in artifacts that are apparently benign to the system and to a reasonable user: the content is treated as ordinary data and typically passes pointwise safety filters because it is not an overt instruction or policy-violating payload in isolation. ARA exploits this epistemic gap: adversarial intent expressed in forms that remain locally admissible until additional context (incident response, provenance, governance evaluation) makes the trajectory’s harmfulness legible.

Definition A.8 (Adversarial Placement). An input is *adversarially placed* if a threat actor deliberately positions apparently benign content knowing that a reasoning model will discover, incorporate, and act on it in ways that serve the threat actor’s objectives. The content itself is apparently benign; the placement is adversarial. For ARA, adversarial placement of apparently benign content is strictly distinguished from overt adversarial input.

A.4 Observational Asymmetry

Definition A.9 (Observational Asymmetry). The model and the human user operate on different projections of the input. The model ingests the full input and updates its internal state. The human observes only a lossy rendering. Information that conditions the model’s state may be imperceptible to the human. The asymmetry runs both directions: the model knows nothing about the governance regime the user operates under.

A.5 Reasoning as Attack Surface

Definition A.10 (Reasoning Capability). A reasoning model engages in extended chains of inference: forming hypotheses, testing them against evidence, refining interpretations. This capability transforms the model from a passive describer into an active searcher. Reasoning capability is not strictly required by ARA, but it is a force multiplier.

A.6 Autonomous Engagement and Chain-of-Thought Opacity

Definition A.11 (Autonomous Engagement). A reasoning model may initiate analysis without explicit instruction. When such a model encounters content that appears to contain structure, its training disposes it to investigate.

Definition A.12 (Chain-of-Thought Opacity). In many deployments, the model’s chain-of-thought is not exposed to users. The process by which the model discovers and acts on data is invisible to human oversight. CoT opacity eliminates the only observational channel through which a human could detect steering in progress.

A.7 Governance and Trajectories

Definition A.13 (Ambient Input). An *ambient input* is any input incorporated into the model’s context that is: (a) not explicitly framed as an instruction, (b) treated as data, evidence, or environment, and (c) nevertheless capable of conditioning internal state.

Definition A.14 (Governance Regime). A *governance regime* $\mathcal{G}$ is an external rule system that defines admissibility of interactional trajectories when evaluated post hoc. A governance regime is one to which the operator, user, or an affected party is actually accountable: laws, regulations,

institutional policies, contractual obligations, professional standards, or platform terms of service that carry real consequences for violation.

Definition A.15 (Trajectory). A *trajectory* $\tau = (h_0, h_1, \dots, h_T)$ is a realized sequence of states arising from a sequence of inputs and controller actions. Associated with each trajectory is an output transcript.

A.8 Steering and Attack

Definition A.16 (Steering). *Steering* is the process by which input x_t affects h_{t+1} on R . This is what inputs do.

Definition A.17 (Ambient Reachability Attack). An *Ambient Reachability Attack* is a situation in which:

1. A threat actor adversarially places apparently benign information in ambient content.
2. A correctly-functioning model discovers this information through its normal analytical processes, operating within the bounds of builder-imposed safety constraints.
3. Steering occurs.
4. An external agent (human or automated) acts on model outputs, or the model acts autonomously.
5. Steps 2–4 repeat and terminate based on the user’s actions.
6. The resulting trajectory τ , when evaluated post hoc, satisfies $\mathcal{G}(\tau) = \text{violation}$ for a governance regime $\mathcal{G}$ to which the operator, user, or an affected party is actually accountable.
7. No individual input in the trajectory is overt adversarial. This property is invariant: it holds during and after τ . The model behaves within K_ϕ at all times.

Remark A.2. We define the trajectory τ as an attack because that is the subject of this paper. However, the same trajectory may be benign under one governance regime and a violation under another. “Attack” is a relational property: it exists in the pair $(\tau, \mathcal{G})$, not in τ alone.